

```

#
=====
# ISE 503 - Health Analytics | Spring 2026
# Project: Predicting High-Cost Medicare Patients Before They Become
High-Cost
# Team: Vansh Chanchlani & Sagarika Raju
#
=====

# — IMPORTS

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (roc_auc_score, classification_report,
                             roc_curve, precision_recall_curve,
                             average_precision_score,
                             confusion_matrix)
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
import xgboost as xgb
import shap
import warnings
warnings.filterwarnings('ignore')

# Plot styling
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12
sns.set_style('whitegrid')
sns.set_palette('muted')

print("✓ All libraries loaded successfully")
✓ All libraries loaded successfully

# — LOAD BENEFICIARY FILES

bene_2022 = pd.read_csv('../datasets/beneficiary_2022.csv', sep='|')
bene_2023 = pd.read_csv('../datasets/beneficiary_2023.csv', sep='|')

print("Beneficiary 2022 shape:", bene_2022.shape)
print("Beneficiary 2023 shape:", bene_2023.shape)

# — LOAD CLAIMS FILES

```

```

print("\nLoading claims files...")

inp_full = pd.read_csv('../datasets/inpatient.csv', sep='|',
                        usecols=['BENE_ID', 'CLM_FROM_DT',
                                'CLM_PMT_AMT',
                                'CLM_TOT_CHRG_AMT',
                                'CLM_UTLZTN_DAY_CNT'])

out_full = pd.read_csv('../datasets/outpatient.csv', sep='|',
                        usecols=['BENE_ID', 'CLM_FROM_DT',
                                'CLM_PMT_AMT',
                                'CLM_TOT_CHRG_AMT'])

car_full = pd.read_csv('../datasets/carrier.csv', sep='|',
                        usecols=['BENE_ID', 'CLM_FROM_DT',
                                'CLM_PMT_AMT'])

# Parse dates and extract year
for df in [inp_full, out_full, car_full]:
    df['CLM_FROM_DT'] = pd.to_datetime(df['CLM_FROM_DT'],
    errors='coerce')
    df['YEAR'] = df['CLM_FROM_DT'].dt.year

print("✓ Inpatient:", inp_full.shape)
print("✓ Outpatient:", out_full.shape)
print("✓ Carrier:", car_full.shape)
print("\nClaims years available:",
sorted(inp_full['YEAR'].dropna().unique()))

Beneficiary 2022 shape: (8671, 185)
Beneficiary 2023 shape: (9179, 185)

Loading claims files...
✓ Inpatient: (58066, 6)
✓ Outpatient: (575092, 5)
✓ Carrier: (1121004, 4)

Claims years available: [2015, 2016, 2017, 2018, 2019, 2020, 2021,
2022, 2023]

# — BUILD YEAR 1 (2022) AND YEAR 2 (2023) SPENDING

```

```

def get_spending_by_year(inp, out, car, year):
    """Aggregate total Medicare payments per beneficiary for a given
    year."""

    inp_yr = inp[inp['YEAR'] == year].groupby('BENE_ID').agg(
        INP_PMT=('CLM_PMT_AMT', 'sum'),

```

```

        INP_STAYS=('CLM_PMT_AMT', 'count'),
        INP_DAYS=('CLM_UTLZTN_DAY_CNT', 'sum')
    ).reset_index()

    out_yr = out[out['YEAR'] == year].groupby('BENE_ID').agg(
        OUT_PMT=('CLM_PMT_AMT', 'sum'),
        OUT_VISITS=('CLM_PMT_AMT', 'count')
    ).reset_index()

    car_yr = car[car['YEAR'] == year].groupby('BENE_ID').agg(
        CAR_PMT=('CLM_PMT_AMT', 'sum'),
        CAR_CLAIMS=('CLM_PMT_AMT', 'count')
    ).reset_index()

    spending = inp_yr.merge(out_yr, on='BENE_ID', how='outer')\
        .merge(car_yr, on='BENE_ID', how='outer')
    spending = spending.fillna(0)
    spending['TOTAL_PMT'] = spending['INP_PMT'] + spending['OUT_PMT']
+ spending['CAR_PMT']
    return spending

# Build spending for Year 1 and Year 2
spend_2022 = get_spending_by_year(inp_full, out_full, car_full, 2022)
spend_2023 = get_spending_by_year(inp_full, out_full, car_full, 2023)

print("Year 1 (2022) beneficiaries with claims:", len(spend_2022))
print("Year 2 (2023) beneficiaries with claims:", len(spend_2023))

print("\nYear 1 spending summary:")
print(spend_2022['TOTAL_PMT'].describe().round(2))

print("\nYear 2 spending summary:")
print(spend_2023['TOTAL_PMT'].describe().round(2))

# — CREATE HIGH-COST LABEL FROM YEAR 2
threshold = spend_2023['TOTAL_PMT'].quantile(0.95)
spend_2023['HIGH_COST'] = (spend_2023['TOTAL_PMT'] >=
threshold).astype(int)

print(f"\nTop 5% spending threshold (Year 2): ${threshold:,.2f}")
print("High cost patients (1):", spend_2023['HIGH_COST'].sum())
print("Non high cost patients (0):",
(spend_2023['HIGH_COST']==0).sum())
print("\nClass distribution:")
print(spend_2023['HIGH_COST'].value_counts(normalize=True).round(3))

Year 1 (2022) beneficiaries with claims: 8301
Year 2 (2023) beneficiaries with claims: 3494

```

```
Year 1 spending summary:
count      8301.00
mean       85401.79
std        500934.23
min         0.00
25%        11223.50
50%        21719.90
75%        60013.50
max       19454109.68
Name: TOTAL_PMT, dtype: float64
```

```
Year 2 spending summary:
count      3494.00
mean       35375.67
std       197069.61
min         0.00
25%        4688.66
50%       13519.76
75%       24700.57
max      8864826.79
Name: TOTAL_PMT, dtype: float64
```

```
Top 5% spending threshold (Year 2): $85,797.90
High cost patients (1): 175
Non high cost patients (0): 3319
```

```
Class distribution:
HIGH_COST
0      0.95
1      0.05
Name: proportion, dtype: float64
```

— MERGE BENEFICIARY DEMOGRAPHICS WITH SPENDING

```
# Select useful demographic columns from beneficiary 2022
demo_cols = ['BENE_ID', 'AGE_AT_END_REF_YR', 'SEX_IDENT_CD',
             'BENE_RACE_CD',
             'ESRD_IND', 'DUAL_ELGBL_MONS', 'BENE_HI_CVRAGE_TOT_MONS',
             'BENE_SMI_CVRAGE_TOT_MONS', 'BENE_HMO_CVRAGE_TOT_MONS',
             'BENE_STATE_BUYIN_TOT_MONS', 'RTI_RACE_CD']

bene_demo = bene_2022[demo_cols].copy()

# Merge Year 1 spending with demographics
df_y1 = spend_2022.merge(bene_demo, on='BENE_ID', how='inner')

# Merge with Year 2 label
label = spend_2023[['BENE_ID', 'HIGH_COST', 'TOTAL_PMT']].rename(
    columns={'TOTAL_PMT': 'Y2_TOTAL_PMT'})
```

```

df_final = df_y1.merge(label, on='BENE_ID', how='inner')

print("Final dataset shape:", df_final.shape)
print("\nClass distribution in final dataset:")
print(df_final['HIGH_COST'].value_counts())
print("\nMissing values:")
print(df_final.isnull().sum()[df_final.isnull().sum() > 0])
print("\nSample:")
print(df_final.head(3).to_string())

```

Final dataset shape: (3339, 21)

Class distribution in final dataset:

HIGH_COST

0 3172

1 167

Name: count, dtype: int64

Missing values:

Series([], dtype: int64)

Sample:

	BENE_ID	INP_PMT	INP_STAYS	INP_DAYS	OUT_PMT	OUT_VISITS
CAR_PMT	CAR_CLAIMS	TOTAL_PMT	AGE_AT_END	REF_YR	SEX_IDENT_CD	
BENE_RACE_CD	ESRD_IND	DUAL_ELGBL_MONS	BENE_HI_CVRAGE_TOT_MONS			
BENE_SMI_CVRAGE_TOT_MONS	BENE_HMO_CVRAGE_TOT_MONS					
BENE_STATE	BUYIN_TOT_MONS	RTI_RACE_CD	HIGH_COST	Y2_TOTAL_PMT		
0	-10000010287996	0.00	0.0	0.0	125687.59	112.0
46881.82		23.0	172569.41		55	2
5	Y		0		12	
12			0		12	5
0		25150.04				
1	-10000010287992	166.97	1.0	1.0	1365.04	2.0
29496.80		16.0	31028.81		83	1
1	0		0		12	
12			0		12	1
0		682.52				
2	-10000010287978	166.97	1.0	1.0	81800.37	51.0
233805.04		156.0	315772.38		69	1
5	0		0		12	
12			0		12	5
0		24649.66				

— EDA: FIGURE 1 - YEAR 2 SPENDING DISTRIBUTION

```

import os
os.makedirs('../outputs', exist_ok=True)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

```

```

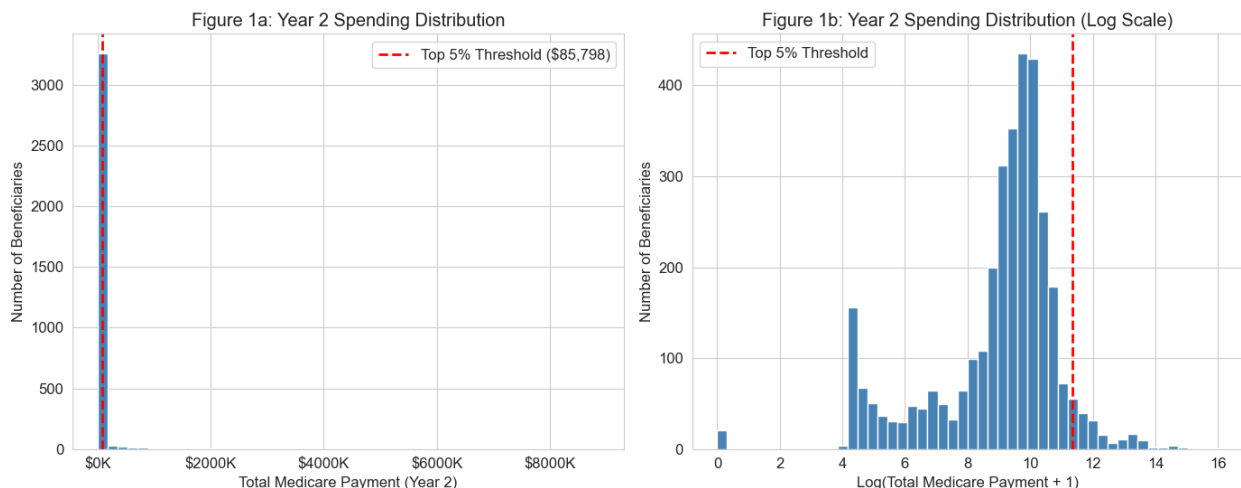
# Left: full distribution
axes[0].hist(df_final['Y2_TOTAL_PMT'], bins=50, color='steelblue',
edgecolor='white')
axes[0].axvline(df_final['Y2_TOTAL_PMT'].quantile(0.95),
                color='red', linestyle='--', linewidth=2, label='Top
5% Threshold ($85,798)')
axes[0].set_xlabel('Total Medicare Payment (Year 2)')
axes[0].set_ylabel('Number of Beneficiaries')
axes[0].set_title('Figure 1a: Year 2 Spending Distribution')
axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x/1000:.0f}K'))
axes[0].legend()

# Right: log scale for better visibility
axes[1].hist(np.log1p(df_final['Y2_TOTAL_PMT']), bins=50,
color='steelblue', edgecolor='white')
axes[1].axvline(np.log1p(df_final['Y2_TOTAL_PMT'].quantile(0.95)),
                color='red', linestyle='--', linewidth=2, label='Top
5% Threshold')
axes[1].set_xlabel('Log(Total Medicare Payment + 1)')
axes[1].set_ylabel('Number of Beneficiaries')
axes[1].set_title('Figure 1b: Year 2 Spending Distribution (Log
Scale)')
axes[1].legend()

plt.suptitle('Year 2 Medicare Spending Distribution', fontsize=14,
fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure1_spending_distribution.png', dpi=150,
bbox_inches='tight')
plt.show()
print("✓ Figure 1 saved")

```

Year 2 Medicare Spending Distribution



✓ Figure 1 saved

— EDA: FIGURE 2 - YEAR 1 SPENDING BY HIGH-COST STATUS

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

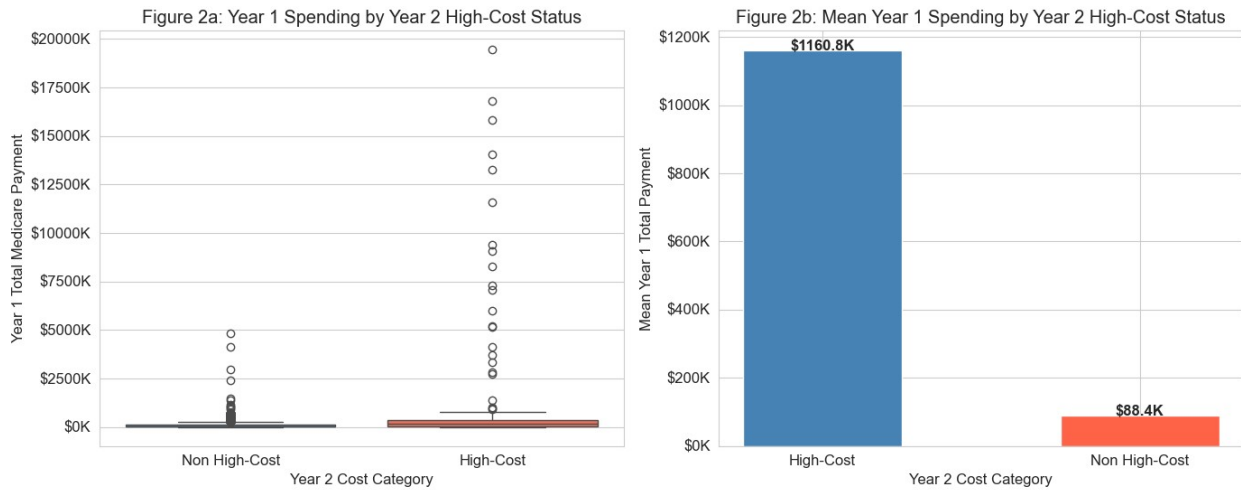
# Left: boxplot
df_final['HIGH_COST_LABEL'] = df_final['HIGH_COST'].map({0: 'Non High-
Cost', 1: 'High-Cost'})

sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='TOTAL_PMT',
            palette={'Non High-Cost': 'steelblue', 'High-Cost':
            'tomato'},
            ax=axes[0])
axes[0].set_xlabel('Year 2 Cost Category')
axes[0].set_ylabel('Year 1 Total Medicare Payment')
axes[0].set_title('Figure 2a: Year 1 Spending by Year 2 High-Cost
Status')
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x/1000:.0f}K'))

# Right: mean spending comparison bar chart
mean_spend = df_final.groupby('HIGH_COST_LABEL')
['TOTAL_PMT'].mean().reset_index()
bars = axes[1].bar(mean_spend['HIGH_COST_LABEL'],
mean_spend['TOTAL_PMT'],
                    color=['steelblue', 'tomato'], edgecolor='white',
width=0.5)
axes[1].set_xlabel('Year 2 Cost Category')
axes[1].set_ylabel('Mean Year 1 Total Payment')
axes[1].set_title('Figure 2b: Mean Year 1 Spending by Year 2 High-Cost
Status')
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x/1000:.0f}K'))
for bar, val in zip(bars, mean_spend['TOTAL_PMT']):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() +
1000,
                f'${val/1000:.1f}K', ha='center', fontweight='bold')

plt.suptitle('Year 1 Spending Patterns by Future High-Cost Status',
            fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure2_spending_by_label.png', dpi=150,
bbox_inches='tight')
plt.show()
print("✓ Figure 2 saved")
```

Year 1 Spending Patterns by Future High-Cost Status



✓ Figure 2 saved

— EDA: FIGURE 3 - DEMOGRAPHIC ANALYSIS

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Left: Age distribution by high-cost status
sns.kdeplot(data=df_final[df_final['HIGH_COST']==0],
            x='AGE_AT_END_REF_YR', ax=axes[0],
            label='Non High-Cost', color='steelblue', fill=True,
            alpha=0.4)
sns.kdeplot(data=df_final[df_final['HIGH_COST']==1],
            x='AGE_AT_END_REF_YR', ax=axes[0],
            label='High-Cost', color='tomato', fill=True, alpha=0.4)
axes[0].set_xlabel('Age at End of Reference Year')
axes[0].set_ylabel('Density')
axes[0].set_title('Figure 3a: Age Distribution\nby High-Cost Status')
axes[0].legend()

# Middle: Sex breakdown
sex_map = {1: 'Male', 2: 'Female'}
df_final['SEX_LABEL'] = df_final['SEX_IDENT_CD'].map(sex_map)
sex_pct = df_final.groupby('SEX_LABEL')
['HIGH_COST'].mean().reset_index()
sex_pct.columns = ['Sex', 'High_Cost_Rate']
bars = axes[1].bar(sex_pct['Sex'], sex_pct['High_Cost_Rate']*100,
                  color=['steelblue', 'tomato'], edgecolor='white',
                  width=0.4)
axes[1].set_xlabel('Sex')
axes[1].set_ylabel('High-Cost Rate (%)')
axes[1].set_title('Figure 3b: High-Cost Rate by Sex')
for bar, val in zip(bars, sex_pct['High_Cost_Rate']):
    axes[1].text(bar.get_x() + bar.get_width()/2,
```



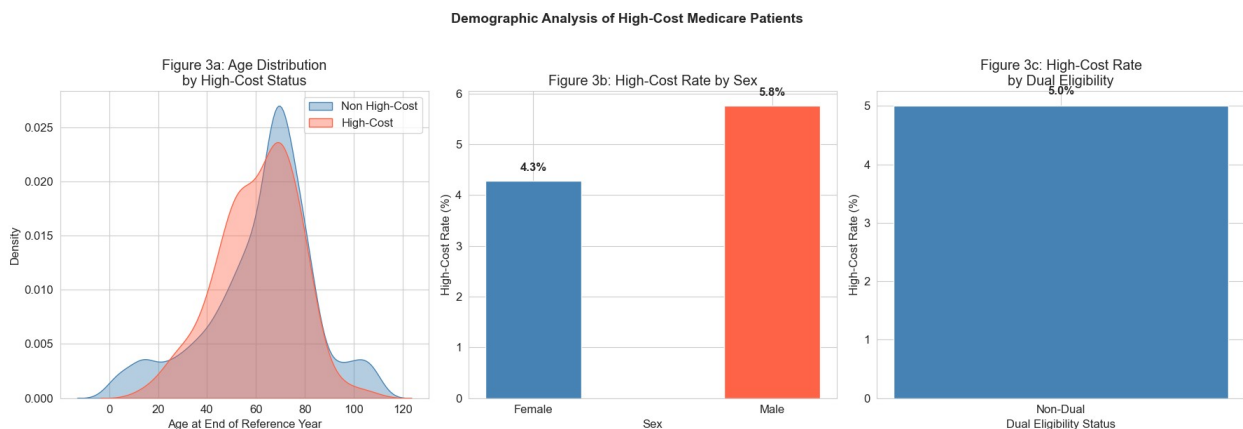
```

        bar.get_height() + 0.2,
        f'{val*100:.1f}%', ha='center', fontweight='bold')

# Right: Dual eligibility
df_final['DUAL_FLAG'] = (df_final['DUAL_ELGBL_MONS'] > 0).map(
    {True: 'Dual Eligible', False: 'Non-Dual'})
dual_pct = df_final.groupby('DUAL_FLAG')
['HIGH_COST'].mean().reset_index()
bars2 = axes[2].bar(dual_pct['DUAL_FLAG'], dual_pct['HIGH_COST']*100,
                    color=['steelblue', 'tomato'], edgecolor='white',
                    width=0.4)
axes[2].set_xlabel('Dual Eligibility Status')
axes[2].set_ylabel('High-Cost Rate (%)')
axes[2].set_title('Figure 3c: High-Cost Rate\nby Dual Eligibility')
for bar, val in zip(bars2, dual_pct['HIGH_COST']):
    axes[2].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.2,
                 f'{val*100:.1f}%', ha='center', fontweight='bold')

plt.suptitle('Demographic Analysis of High-Cost Medicare Patients',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure3_demographics.png', dpi=150,
            bbox_inches='tight')
plt.show()
print("✓ Figure 3 saved")

```



✓ Figure 3 saved

— EDA: FIGURE 4 - UTILIZATION PATTERNS

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
```

Left: Inpatient stays by high-cost status

```
sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='INP_STAYS',
            palette={'Non High-Cost': 'steelblue', 'High-Cost':
```

```

'tomato'},
    ax=axes[0], showfliers=False)
axes[0].set_xlabel('Year 2 Cost Category')
axes[0].set_ylabel('Number of Inpatient Claims (Year 1)')
axes[0].set_title('Figure 4a: Inpatient Utilization\nby High-Cost
Status')

# Middle: Outpatient visits
sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='OUT_VISITS',
    palette={'Non High-Cost': 'steelblue', 'High-Cost':
'tomato'},
    ax=axes[1], showfliers=False)
axes[1].set_xlabel('Year 2 Cost Category')
axes[1].set_ylabel('Number of Outpatient Claims (Year 1)')
axes[1].set_title('Figure 4b: Outpatient Utilization\nby High-Cost
Status')

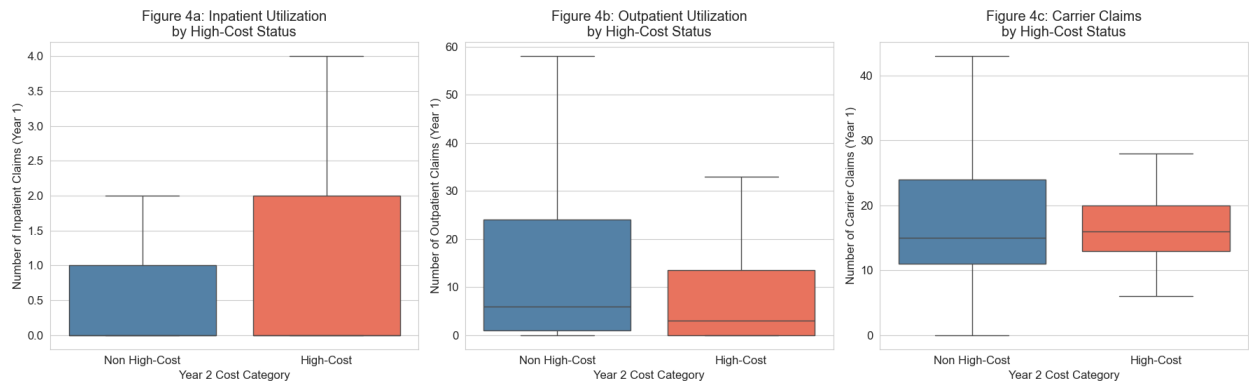
# Right: Carrier claims
sns.boxplot(data=df_final, x='HIGH_COST_LABEL', y='CAR_CLAIMS',
    palette={'Non High-Cost': 'steelblue', 'High-Cost':
'tomato'},
    ax=axes[2], showfliers=False)
axes[2].set_xlabel('Year 2 Cost Category')
axes[2].set_ylabel('Number of Carrier Claims (Year 1)')
axes[2].set_title('Figure 4c: Carrier Claims\nby High-Cost Status')

plt.suptitle('Year 1 Utilization Patterns by Future High-Cost Status',
    fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../outputs/figure4_utilization.png', dpi=150,
bbox_inches='tight')
plt.show()

# Print mean utilization stats
print("Mean utilization by high-cost status:")
print(df_final.groupby('HIGH_COST_LABEL')
[['INP_STAYS', 'OUT_VISITS', 'CAR_CLAIMS']].mean().round(1))
print("✓ Figure 4 saved")

```

Year 1 Utilization Patterns by Future High-Cost Status



Mean utilization by high-cost status:

	INP_STAYS	OUT_VISITS	CAR_CLAIMS
HIGH_COST_LABEL			
High-Cost	10.3	16.8	30.3
Non High-Cost	2.0	23.3	30.0

✓ Figure 4 saved

— CELL 9: BASELINE LOGISTIC REGRESSION

Feature columns

```
feature_cols = ['TOTAL_PMT', 'INP_PMT', 'OUT_PMT', 'CAR_PMT',
                'INP_STAYS', 'INP_DAYS', 'OUT_VISITS', 'CAR_CLAIMS',
                'AGE_AT_END_REF_YR', 'SEX_IDENT_CD', 'BENE_RACE_CD',
                'BENE_HI_CVRAGE_TOT_MONS', 'BENE_SMI_CVRAGE_TOT_MONS',
                'BENE_HMO_CVRAGE_TOT_MONS']
```

```
X = df_final[feature_cols]
```

```
y = df_final['HIGH_COST']
```

Train/test split (70/30, stratified)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

Scale features

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

Apply SMOTE to handle class imbalance

```
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train_sc, y_train)
print("After SMOTE - Class distribution:")
print(pd.Series(y_train_sm).value_counts())
```

Fit Logistic Regression

```
lr = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
```

```

lr.fit(X_train_sm, y_train_sm)

# Evaluate
y_pred_proba = lr.predict_proba(X_test_sc)[:, 1]
auc = roc_auc_score(y_test, y_pred_proba)
print(f"\n✓ Logistic Regression AUC-ROC: {auc:.4f}")

# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(fpr, tpr, color='steelblue', lw=2,
        label=f'Logistic Regression (AUC = {auc:.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random Classifier')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('Figure 5: ROC Curve - Logistic Regression Baseline')
ax.legend(loc='lower right')
plt.tight_layout()
plt.savefig('../outputs/figure5_roc_curve.png', dpi=150,
            bbox_inches='tight')
plt.show()

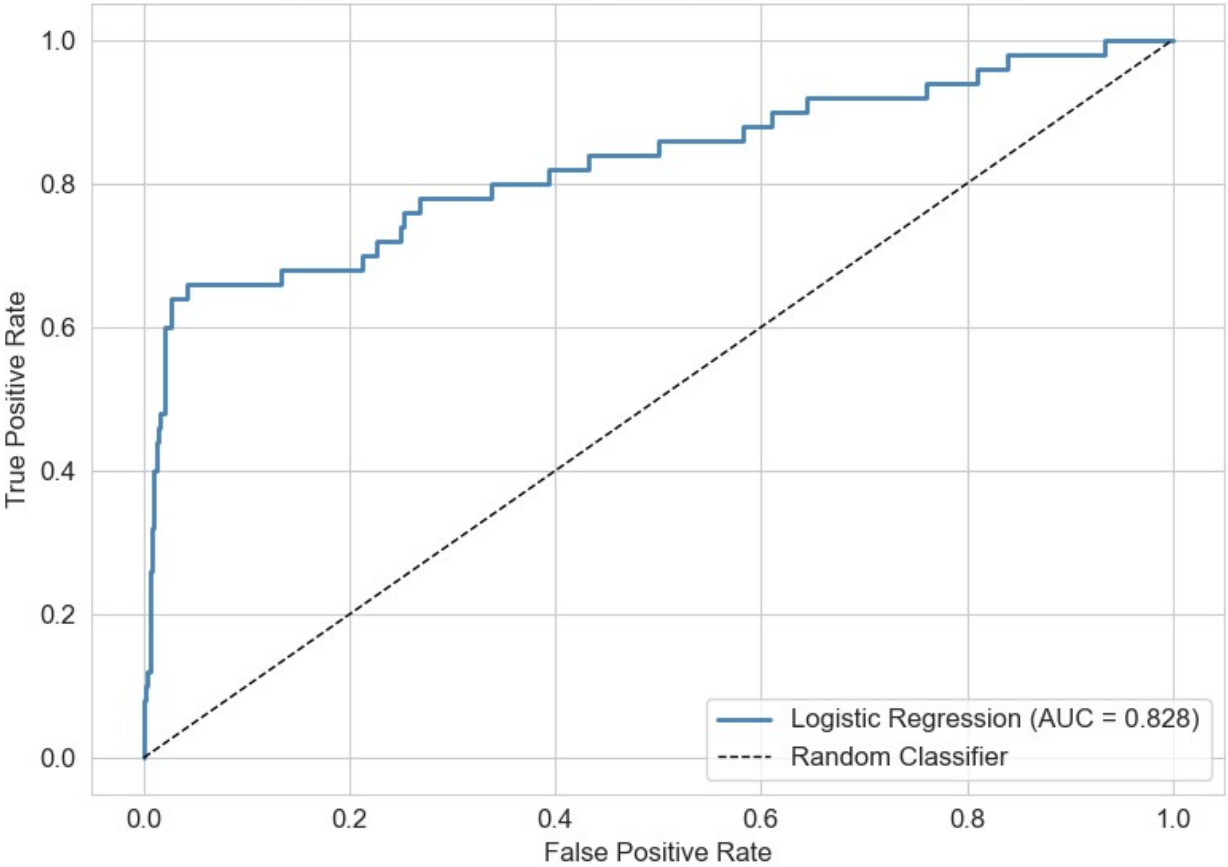
print("\nClassification Report:")
y_pred = lr.predict(X_test_sc)
print(classification_report(y_test, y_pred, target_names=['Non High-
Cost', 'High-Cost']))
print("✓ Figure 5 saved")

After SMOTE - Class distribution:
HIGH_COST
0      2220
1      2220
Name: count, dtype: int64

✓ Logistic Regression AUC-ROC: 0.8283

```

Figure 5: ROC Curve - Logistic Regression Baseline



Classification Report:

	precision	recall	f1-score	support
Non High-Cost	0.98	0.96	0.97	952
High-Cost	0.44	0.64	0.52	50
accuracy			0.94	1002
macro avg	0.71	0.80	0.75	1002
weighted avg	0.95	0.94	0.95	1002

✓ Figure 5 saved